Autonomous Racing Agent using PPO based Reinforcement Learning in Simulated Environment

Abstract

This project focuses on developing an autonomous racing car agent using reinforcement learning. The work was carried out on a modified version of the Gymnasium CarRacing simulator originally developed by OpenAI. Instead of following fixed driving rules, the agent learns steering, acceleration, and braking through repeated interaction with the environment. A custom implementation of the Proximal Policy Optimization (PPO) algorithm was used for training.

Several improvements were made to the simulator, including a custom reward system, centerline guidance, smoother steering control, and corner awareness. The agent received rewards for forward progress and staying on track, while penalties were given for off-road movement and unstable driving. These changes helped improve the learning process and vehicle control.

The trained model was evaluated using total reward, track completion percentage, and lap completion status. Results showed gradual improvement in navigation and driving performance over time. This project demonstrates the effectiveness of reinforcement learning for autonomous racing in simulated environments.

1. Main Objectives

- Develop an autonomous racing agent using reinforcement learning.
- Train the agent using a custom PPO algorithm for driving control.
- Modify the Gymnasium CarRacing environment for better learning performance.
- Evaluate the agent using reward, track completion, and lap success metrics.
- Build a real-time simulation for training and testing the agent.

2. Status and Other Details: - Completed

- Percentage contribution of members: P Rohith (100%)
- Total time spent on project : 5 Weeks

3. Major Stumbling Blocks:

- Initial training produced poor results, with low reward, low track completion, and frequent off-track movement.
- Designing an effective reward function, tuning PPO hyperparameters, and repeated long training runs made the project time-consuming.

4. Introduction:

Autonomous racing is a challenging research problem in artificial intelligence, robotics, and intelligent transportation systems. It combines perception, real-time decision making, path planning, and continuous vehicle control in a dynamic environment. A racing vehicle must learn how to steer accurately, control throttle, apply braking at the correct time, and remain stable while handling sharp turns and changing track layouts. Traditional rule-based systems often struggle in such environments because manually defining driving behaviour for every possible situation is difficult. Reinforcement learning (RL) offers a more flexible solution, where an agent learns optimal behaviour through repeated interaction with the environment and by maximizing cumulative reward [3].

Reinforcement learning has a long history in sequential decision making. Foundational methods such as Dynamic Programming and value-based learning established the theoretical basis for learning control policies from reward feedback [3]. However, classical RL methods were limited when dealing with high-dimensional inputs such as images. Recent advances in deep learning [4] and deep reinforcement learning [9] have solved this limitation by combining neural networks with RL algorithms. This has enabled agents to learn directly from raw observations and achieve strong performance in games, robotics, and

control tasks. These developments have made RL highly suitable for autonomous driving and racing applications.

Continuous vehicle control requires smooth and stable actions rather than simple discrete commands. Modern policy-gradient methods such as Proximal Policy Optimization (PPO) were developed to address continuous action problems. Among them, PPO is widely adopted because of its stable updates, efficient learning process, and practical implementation. PPO uses a clipped objective function to avoid excessively large policy changes during training, which improves convergence and stability [1]. Due to these advantages, PPO was selected as the learning algorithm for this project.

In this project, an autonomous racing agent was developed using a custom implementation of PPO. The complete training and testing pipeline was coded from scratch using PyTorch [5], without depending on ready-made training scripts. Neural network models, rollout collection, policy updates, checkpoint saving, evaluation, and testing modules were implemented and customized specifically for the racing task. This provided better control over experimentation, hyperparameter tuning, and environment interaction.

Training was carried out using the Gymnasium CarRacing simulator, originally developed from the OpenAI Gym benchmark suite [2], [6], [7]. This environment provides a top-down racing track generated randomly for each episode. The agent receives observations from the environment and must output continuous steering, acceleration, and brake commands. The CarRacing task is widely recognized as a useful benchmark because it combines high-dimensional visual inputs with complex control behaviour. Training in simulation also provides major advantages such as lower cost, faster experimentation, repeatability, and zero physical risk compared to real-world vehicle testing [9].

Significant modifications were made to the original environment to improve training performance. The base simulator contained approximately 800 lines of code, which was extended to nearly 1700 lines after customization. Several new features were introduced, including custom reward shaping, centerline guidance, smoother steering control, path-angle observations, corner awareness, lap completion tracking, improved termination conditions, and additional performance statistics. These modifications were designed after observing poor early training behaviour such as off-track movement, unstable steering, and low track completion. Reward engineering played an important role by encouraging forward progress, staying near the centerline, and smoother driving while penalizing off-road movement and poor alignment.

The final system was evaluated using multiple performance metrics such as total cumulative reward, percentage of track completed, average completion across episodes, and lap completion rate. These measures indicate how effectively the agent can navigate the track and maintain control over time. Experimental results showed clear improvement during training, where the agent gradually learned better steering behaviour, smoother turning, and longer track completion.

This project demonstrates that deep reinforcement learning methods, particularly PPO, can successfully learn autonomous racing behaviour in simulated environments [1], [9]. It also highlights the importance of custom environment design, reward shaping, and algorithm implementation in improving learning outcomes. The proposed system can be further extended for more realistic simulators, obstacle avoidance tasks, and real-world autonomous driving research.

5. Background Theory

5.1 Reinforcement Learning for Autonomous Racing Environment

Reinforcement Learning (RL) is a machine learning method in which an agent learns how to make decisions by interacting with an environment. Unlike supervised learning, RL does not use labeled input-output data. Instead, the agent improves through trial and error by receiving rewards or penalties after every action. The main objective is to learn an optimal control policy that maximizes long-term cumulative reward. RL is widely used in

robotics, game playing, process control, and autonomous driving systems.

In this project, the autonomous racing vehicle acts as the agent, while the modified Gymnasium CarRacing simulator acts as the environment. At each time step t, the agent observes the current state s_t, selects an action a_t, receives reward r_t, and moves to the next state s_{t+1}. This interaction can be represented as:

$$(s_t, a_t, r_t, s_{t+1})$$

or

$$s_t \xrightarrow{a_t} s_{t+1}$$

The goal is to learn a policy πsuch that:

$$a_t = \pi(s_t)$$

where πmaps each state to the most suitable action.

Markov Decision Process (MDP)

The reinforcement learning problem is commonly modeled as a Markov Decision Process (MDP) defined by:

$$(S, A, P, R, \gamma)$$

where:

- S= set of states
- A= set of actions
- P= state transition probability
- R= reward function
- γ= discount factor

For this project:

- State = vehicle and track observations
- Action = steering, throttle, brake
- Reward = progress-based custom reward
- Discount factor = future reward importance

State Representation

The state vector in this project was improved through custom environment modification. Instead of using only raw image observations, additional features such as path angle, centerline position, track alignment, and vehicle motion were included.

Thus:

$$s_t = [x_t, y_t, \theta_t, v_t, d_t, c_t]$$

where:

- x_t, y_t= vehicle position
- θ_t= heading angle
- v_t= speed
- d_t= centerline distance
- c_t= curvature / corner feature

This richer observation space improved training performance.

Action Space

The racing environment uses continuous control actions:

$$a_t = [u_1, u_2, u_3]$$

where:

$$u_1 = \text{steering}, u_2 = \text{throttle}, u_3 = \text{brake}$$

with limits:

$$-1 \leq \text{steering} \leq 1$$
$$0 \leq \text{throttle} \leq 1$$
$$0 \leq \text{brake} \leq 1$$

Continuous actions are more suitable for driving because steering and acceleration need smooth adjustments.

Reward Function

The reward function guides the learning behaviour of the agent. In this project, custom reward shaping was used:

$$r_t = r_{progress} + r_{track} + r_{smooth} - r_{offroad} - r_{error}$$

where:

- $r_{progress}$= reward for moving forward
- r_{track} = reward for staying near centerline
- r_{smooth}= reward for stable motion
- $r_{offroad}$= penalty for leaving road
- r_{error}= penalty for wrong direction or oscillation

A simplified version:

$$r_t = \alpha p_t + \beta c_t - \lambda o_t$$

where:

- p_t= progress
- c_t= center alignment
- o_t= off-road indicator

This reward design helped overcome poor early training behaviour.

Return and Objective

The cumulative reward for one episode is:

$$R = \sum_{t=0}^{T} r_t$$

The discounted return is:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

or

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The learning objective is:

$$\max_{\pi} \mathbb{E}[G_t]$$

which means the policy should maximize expected long-term reward.

Value Functions

The state value function is:

$$V^{\pi}(s) = \mathbb{E}[G_t \mid s_t = s]$$

The action value function is:

$$Q^{\pi}(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]$$

These functions estimate how good a state or action is during training.

Episodes in This Project

One complete race attempt is called an episode. An episode ends when:

- Lap is completed
- Vehicle goes off-track
- Maximum step limit reached
- Invalid motion detected

The model was trained over many episodes to improve completion percentage and lap success.

5.2 Custom Proximal Policy Optimization (PPO) Algorithm

Proximal Policy Optimization (PPO) is an advanced policy-gradient reinforcement learning algorithm used for solving continuous control problems. It is widely applied in robotics, gaming, and autonomous systems because it provides stable learning while maintaining good sample efficiency. In this project, PPO was selected to train the autonomous racing vehicle since the task requires smooth steering, acceleration, and braking control.

Unlike value-based methods such as Q-learning, PPO directly learns a policy that maps

states to actions. The objective is to maximize the expected cumulative reward obtained by the racing agent while driving on the track.

Policy Objective

The policy is represented as:

$$\pi_\theta(a_t \mid s_t)$$

where:

- s_t= current state
- a_t= selected action
- θ= trainable neural network parameters

The optimization objective is:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[R]$$

where Ris the total reward collected during an episode.

PPO Probability Ratio

PPO compares the new policy with the old policy using the ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta old}(a_t \mid s_t)}$$

This ratio measures how much the updated policy changes compared to the previous policy.

Clipped PPO Loss Function

To prevent unstable updates, PPO uses a clipped objective:

$$L^{clip}(\theta) = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)]$$

where:

- A_t= advantage estimate
- ϵ= clipping factor

This clipping prevents very large parameter updates and improves training stability.

Advantage Estimation

The advantage tells whether an action performed better than expected:

$$A_t = G_t - V(s_t)$$

where:

- G_t= actual return
- $V(s_t)$= critic estimated value

In this project, Generalized Advantage Estimation (GAE) was implemented for better learning:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$
$$A_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

where:

- $\gamma = 0.99$
- $\lambda = 0.95$

These values were directly used in my code.

PPO Implementation in This Project

The PPO algorithm was completely coded from scratch using PyTorch. The training pipeline included:

- rollout data collection
- reward storage
- log probability calculation
- GAE computation
- mini-batch optimization
- model checkpoint saving
- best model tracking
- testing framework

The main training settings used in your code were:

- Rollout steps = 2048
- PPO epochs = 10

- Batch size = 256
- Learning rate = 2×10^{-4}
- Clip epsilon = 0.2

These hyperparameters were tuned for better racing performance.

Beta Distribution Action Policy

Since actions are continuous, your PPO model used a Beta probability distribution instead of a normal Gaussian distribution.

$$a_t \sim \text{Beta}(\alpha, \beta)$$

The sampled action was later converted to environment controls:

$$a_t = [\text{steering}, \text{throttle}, \text{brake}]$$

This helped keep outputs within valid action limits.

Total Loss Function Used

The final optimization loss included:

$$\text{Loss} = \text{PolicyLoss} + c_1(\text{ValueLoss}) - c_2(\text{Entropy})$$

where:

- ValueLoss improves critic accuracy
- Entropy encourages exploration
- Gradient clipping was also applied for stability

Why PPO Was Suitable for This Project

PPO was selected because:

- Works well for continuous action spaces
- Stable learning compared to basic policy gradient
- Efficient training in simulation
- Good balance between exploration and exploitation
- Suitable for steering and speed control tasks

5.3 Custom Actor-Critic Neural Network Architecture

The performance of a reinforcement learning agent mainly depends on how effectively it can represent the policy and estimate future rewards. In this project, a custom Actor-Critic neural network was developed using PyTorch to train the autonomous racing agent. The model was designed specifically for the PPO algorithm and continuous vehicle control tasks. It learns how to generate driving actions while also estimating the quality of the current state.

The Actor-Critic method combines two learning components:

- Actor Network – selects steering, throttle, and brake actions.
- Critic Network – estimates the expected future reward of the current state.

Using both components together improves training stability and convergence speed compared to using only policy learning.

Overall Architecture

The network receives the observation vector from the modified CarRacing environment and processes it through shared hidden layers. After feature extraction, the network splits into separate Actor and Critic output heads.

$$\text{Input} \rightarrow \text{Shared Layers} \rightarrow \text{Actor Head} + \text{Critic Head}$$

This shared architecture reduces computation and allows both networks to learn common environment features.

Input Layer

The input to the network is the state observation vector generated from the custom environment. It includes features such as:

- path angle
- track alignment
- vehicle movement information
- centerline guidance
- additional racing state variables

Let the input state be:

$$\boldsymbol{s_t = [x_1, x_2, x_3, \ldots, x_n]}$$

where $\boldsymbol{n}$is the observation dimension.

Shared Hidden Layers

According to your code, the feature extraction backbone consists of two fully connected layers:

$$\boldsymbol{h_1 = tanh(W_1 s_t + b_1)}$$
$$\boldsymbol{h_2 = tanh(W_2 h_1 + b_2)}$$

where:

- Hidden layer size = 128 neurons
- Activation function = Tanh

Thus, your actual architecture is:

- Linear($\boldsymbol{obs\_dim, 128}$)
- Tanh
- Linear(128,128)
- Tanh

The Tanh function helps produce smooth outputs suitable for control problems.

Actor Head

Instead of directly outputting actions, your Actor network predicts parameters of a Beta probability distribution for each control action.

$$\boldsymbol{\alpha = softplus(W_\alpha h_2 + b_\alpha) + 1}$$
$$\boldsymbol{\beta = softplus(W_\beta h_2 + b_\beta) + 1}$$

where:

- $\boldsymbol{\alpha, \beta > 1}$ensure valid Beta parameter
- Separate values are produced for steering, throttle, and brake

The final action is sampled as:

$$\boldsymbol{u_t \sim Beta(\alpha, \beta)}$$

Then converted into environment controls:

$$\boldsymbol{a_t = [steering}\text{,throttle,}\boldsymbol{brake]}$$

For steering:

$$\boldsymbol{steering = 2u - 1}$$

so that steering lies in:

$$\boldsymbol{[-1, 1]}$$

Throttle and brake remain in:

$$\boldsymbol{[0, 1]}$$

This Beta distribution design is directly based on your code and is better suited for bounded control outputs.

Critic Head

The Critic network estimates the value of the current state:

$$\boldsymbol{V(s_t) = W_v h_2 + b_v}$$

This value helps determine how good the current racing position is and is used during PPO updates.

If the vehicle is well aligned and progressing forward, the value estimate becomes higher.

Weight Initialization

Your model uses orthogonal initialization for stable training:

$$\boldsymbol{W^T W = I}$$

This improves gradient flow and helps faster convergence during learning.

Bias values for Beta heads were also carefully initialized to encourage reasonable starting actions.

Forward Pass in the Model

The complete forward operation is:

$$\boldsymbol{s_t \rightarrow h_1 \rightarrow h_2 \rightarrow (\alpha, \beta, V)}$$

Then:

- Actor samples control action
- Critic predicts state value

This allows simultaneous policy learning and value estimation.

Why This Architecture Was Chosen

This custom network was selected because:

- Lightweight and computationally efficient
- Suitable for vector observations
- Stable for PPO training
- Supports continuous bounded actions
- Better than simple direct action output methods

Role in This Project

Initially, the network outputs random actions and poor control. During training, weights are updated repeatedly through PPO optimization. Over time, the Actor learns smoother steering and speed control, while the Critic learns to estimate better racing states.

This helped the vehicle achieve:

- improved track completion
- better corner handling
- smoother movement
- reduced off-track errors

5.4 Environment Modifications and Reward Engineering

The effectiveness of a reinforcement learning agent depends strongly on the quality of the environment used for training. In this project, the original Gymnasium CarRacing simulator was extensively modified to improve observation quality, reward signals, progress tracking, and training efficiency. Direct training on the default environment initially resulted in poor rewards, unstable steering, and frequent off-track failures. Therefore, a custom environment was developed to make the racing task more learnable for the PPO agent.

The original simulator contained approximately **800 lines of code**, which was expanded to nearly **1700 lines** after modification. These changes included custom observations, reward shaping, lap completion logic, completion metrics, and better termination conditions.

Enhanced State Representation

The agent receives a state vector containing useful racing information instead of relying only on default observations.

$$s_t = [x_t, y_t, v_t, \theta_t, \phi_t, d_t, c_t]$$

where:

- x_t, y_t= vehicle position
- v_t= speed
- θ_t= vehicle heading angle
- ϕ_t= track path angle
- d_t= centerline deviation
- c_t= curvature / corner feature

The path-angle error is:

$$e_\theta = \phi_t - \theta_t$$

A small e_θ means the vehicle is aligned with the road direction.

The centerline error is:

$$e_c = | p_{car} - p_{center} |$$

where p_{car} is vehicle lateral position and p_{center} is road center position.

These engineered observations helped the model learn better steering behaviour.

Continuous Control Actions

The racing agent outputs three continuous actions:

$$a_t = [u_s, u_a, u_b]$$

where:

- u_s= steering
- u_a= acceleration
- u_b= brake

with bounds:

$$-1 \le u_s \le 1$$
$$0 \le u_a \le 1$$
$$0 \le u_b \le 1$$

Continuous control allows smoother vehicle motion compared to discrete commands.

Custom Reward Function

Reward shaping was one of the most important environment modifications. The total reward at each step was formed using multiple components:

$$r_t = r_p + r_c + r_s - r_o - r_u$$

where:

- r_p= progress reward
- r_c= centerline reward
- r_s= smooth driving reward
- r_o= off-road penalty
- r_u= unstable motion penalty

A more detailed form can be written as:

$$r_t = \alpha \Delta L_t + \beta(1 - e_c) + \gamma \cos(e_\theta) - \lambda O_t - \mu J_t$$

where:

- ΔL_t= forward progress along track
- e_c= centerline error
- e_θ= heading error
- O_t= off-road indicator (0 or 1)
- J_t= steering jerk / unstable action
- $\alpha, \beta, \gamma, \lambda, \mu$= reward weights

This reward system encouraged efficient and stable racing behaviour.

Progress Measurement

Track completion was measured using visited track tiles:

$$C_t = \frac{N_v}{N_T} \times 100$$

where:

- N_v= number of visited tiles
- N_T= total track tiles

Higher C_t indicates better navigation performance.

Lap Completion Logic

The environment was modified to detect successful full lap completion:

$$Lap = \begin{cases} 1, & C_t = 100\% \\ 0, & C_t < 100\% \end{cases}$$

This metric was important for evaluating final model performance.

Episode Termination Conditions

Episodes ended when one of the following occurred:

$$done = \begin{cases} 1, Lap\ Complete \\ 1, Timeout \\ 1, Severe\ offroad \\ 1, No\ progress \\ 0, otherwise \end{cases}$$

This reduced wasted training time on failed runs.

Steering Smoothness Constraint

To reduce unstable oscillations, steering smoothness was encouraged by penalizing rapid action changes:

$$J_t = | u_s(t) - u_s(t-1) |$$

Lower jerk values lead to smoother turning and more realistic driving.

Effect of Modifications

After applying these environment improvements, the PPO agent gradually learned:

- better lane alignment
- smoother steering actions
- improved corner handling
- higher completion percentage
- more consistent rewards

Without these changes, learning was slow and unstable.

6. Proposed Methodology

The main objective of this project is to develop an autonomous racing agent that can learn efficient driving behaviour without manually programmed rules. Instead of using fixed logic for steering, acceleration, and braking, the proposed system allows the vehicle to learn by interacting with a modified racing simulator. Reinforcement learning was used as the core learning method, where the agent improves its policy through repeated trial-and-error experience.

The complete model was trained using a custom implementation of the Proximal Policy Optimization (PPO) algorithm with an Actor-Critic neural network architecture. The original Gymnasium CarRacing environment was selected as the simulation platform and was significantly modified to improve training quality. The source code of the environment was expanded from nearly 800 lines to around 1700 lines by introducing improved observations, reward engineering, lap completion logic, and better termination conditions.

At each time step t, the racing agent observes the current environment state s_t, predicts a driving action a_t, receives reward r_t, and moves to the next state s_{t+1}. This learning interaction is represented as:

$$(s_t, a_t, r_t, s_{t+1})$$

The aim of the model is to learn a policy π_θ that maximizes the expected cumulative reward:

$$a_t = \pi_\theta(s_t)$$

$$J(\theta) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

where θ represents trainable network parameters and γ is the discount factor.

The observation space of the simulator was improved by including structured state features instead of depending only on default track perception. The final state vector contains useful driving information such as vehicle speed, heading angle, road direction, centerline error, and corner information. The state representation can be written as:

$$s_t = [x_t, y_t, v_t, \theta_t, \phi_t, e_c, c_t]$$

where x_t, y_tdenote position coordinates, $v_t$denotes vehicle speed, $\theta_t$is vehicle heading angle, $\phi_t$is track direction angle, $e_c$is centerline deviation, and $c_t$represents corner curvature information. The heading alignment error is given by:

$$e_\theta = \phi_t - \theta_t$$

This additional information helped the agent make better turning decisions.

The control action generated by the policy consists of three continuous values:

$$a_t = [u_s, u_a, u_b]$$

where $u_s$is steering, $u_a$is throttle, and $u_b$is brake. The valid control limits are:

$$-1 \le u_s \le 1$$
$$0 \le u_a, u_b \le 1$$

This continuous action design provides smoother and more realistic vehicle control than discrete actions.

The neural network used in this project follows an Actor-Critic structure. Shared hidden layers first extract useful features from the state vector, and then the network splits into two outputs. The Actor predicts the driving action, while the Critic estimates the value of the current state. The hidden layers are defined as:

$$h_1 = tanh(W_1 s_t + b_1)$$
$$h_2 = tanh(W_2 h_1 + b_2)$$

The Actor network predicts Beta-distribution parameters $\alpha$and β, and the control action is sampled as:

$$u_t \sim Beta(\alpha, \beta)$$

This approach ensures bounded continuous outputs suitable for steering, throttle, and brake. The Critic estimates:

$$V(s_t) = W_v h_2 + b_v$$

which helps in stable policy updates.

To improve learning stability, PPO clipping was used. The probability ratio between new and old policies is:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta old}(a_t \mid s_t)}$$

The PPO objective function is:

$$L^{clip}(\theta) = \mathbb{E}[\min(rtAt,\ clip(rt, 1-\epsilon, 1+\epsilon)At)]$$

where $A_t$is the advantage estimate. This clipped objective prevents sudden policy changes and improves convergence during training.

Reward engineering was one of the most important contributions of the proposed methodology. The default reward system was redesigned to encourage better racing behaviour. Positive rewards were given for forward progress, staying near the centerline, and smooth motion, while penalties were applied for off-road movement and unstable steering. The reward can be represented as:

$$r_t = r_p + r_c + r_s - r_o - r_u$$

where $r_p$is progress reward, $r_c$is centerline reward, $r_s$is smooth driving reward, $r_o$is off-road penalty, and $r_u$is unstable control penalty.

To evaluate performance, completion percentage was introduced:

$$C = \frac{N_v}{N_T} \times 100$$

where $N_v$is the number of visited track tiles and $N_T$is the total number of track tiles. Lap completion and cumulative episode reward were also recorded:

$$R = \sum_{t=0}^{T} r_t$$

The complete methodology follows these steps: initialize environment, observe current state, predict driving action, execute action, collect reward, store rollout data, update PPO model, save best checkpoint, and repeat until training completion.

Using this proposed methodology, the autonomous racing agent gradually learned smoother steering, better corner handling, improved completion percentage, and more stable driving performance. The system demonstrates that reinforcement learning combined with custom environment engineering can effectively solve autonomous racing problems in simulation.

7. Experimentation

To evaluate the effectiveness of the proposed autonomous racing system, extensive training and testing experiments were carried out using the modified Gymnasium CarRacing environment. The custom PPO algorithm with Actor-Critic neural network was trained from scratch using PyTorch. Performance was analyzed using cumulative reward, track completion percentage, lap completion count, and testing consistency.

The experiments were conducted on a CPU-based system using automatic device selection. The total training timesteps were set to:

$$\mathrm{T} = 1{,}800{,}000$$

A total of 2376 training episodes were completed during learning. The approximate training time was around 1.3 hours.

The trained model was periodically saved using checkpoint logic, and the best-performing model was selected based on reward and completion metrics.

Training Performance

The training progress was evaluated using episode reward and completion percentage.

The cumulative reward for each episode is:

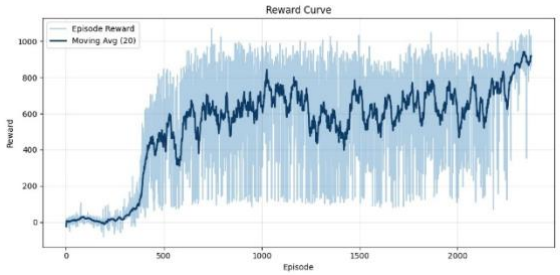
$$\mathrm{R} = \sum_{\mathrm{t}=0}^{\mathrm{N}} \mathrm{r_t}$$

where $\mathrm{r_t}$is reward at step t.

The final training statistics obtained are shown below.

Table 1: Training Results

Parameter	Value
Total Timesteps	1,800,000
Episodes Logged	2376
Final Average Reward (Last 10 Episodes)	939.64
Final Moving Average Reward (20 Episodes)	919.15
Best Reward Achieved	1070.93
Overall Average Completion	64.39%
Final Completion Average (Last Episodes)	100.00%
Best Completion	100.00%
Lap Completed	Yes
Total Laps Completed	982



Reward Curve Analysis

During the early training stage, the reward remained low because the agent explored random actions and frequently moved off-track. After several hundred episodes, a rapid improvement in reward was observed as the model learned better steering and throttle control.

The moving average reward gradually increased and stabilized near:

$$R_{avg} \approx 900$$

with the best reward reaching:

$$R_{max} = 1070.93$$

This indicates successful learning of stable racing behaviour.

Completion Curve Analysis

Track completion percentage was used to measure how much of the racing track the agent successfully navigated.

$$C = \frac{N_v}{N_t} \times 100$$

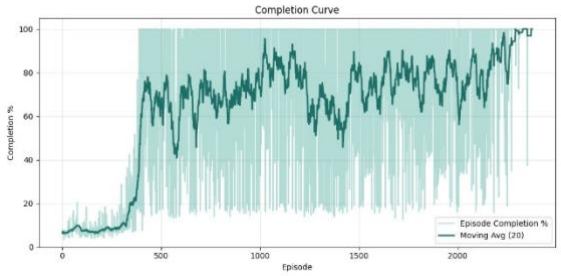
where:

- N_v= visited tiles
- N_t= total tiles in track

Initially, completion remained low due to poor control. As training progressed, completion percentage increased steadily and reached:

$$C = 100\%$$

in the final episodes. This shows that the agent learned to finish complete laps consistently.



Testing Results

After training, the best saved model was tested on unseen tracks for three episodes. The obtained results are given below.

Table 2: Testing Performance

Episode	Reward	Completion	Lap Finished
1	965.29	100.0%	True
2	896.93	100.0%	True
3	947.05	100.0%	True

Average test performance:

$$Reward_{avg} = 936.42$$
$$Completion_{avg} = 100.0\%$$

These results confirm that the trained agent generalizes well and can complete full laps consistently.

Learning Behaviour Observed

From the training and testing experiments, the following improvements were observed:

- smoother steering actions
- better corner handling
- improved throttle control
- reduced off-track movement
- consistent lap completion
- stable high reward values

Discussion

The results clearly show that combining PPO with a modified racing environment significantly improved learning performance. The reward shaping and enhanced observations helped the agent learn faster than standard sparse-reward training. The high final completion rate and repeated successful laps indicate that the model ***developed robust autonomous driving behaviour.***

The successful completion of 982 laps during training further demonstrates strong policy convergence.

8. Conclusion

In this paper, a custom reinforcement learning based approach was proposed for autonomous racing using a modified Gymnasium CarRacing environment. The system was developed using the Proximal Policy Optimization (PPO) algorithm with a custom Actor-Critic neural network implemented from scratch using PyTorch. Unlike traditional rule-based driving systems, the proposed model learns steering, throttle, and braking actions directly through interaction with the environment.

To improve learning performance, the original simulator was significantly modified by introducing enhanced observations, path-angle features, reward shaping, lap completion tracking, and improved termination logic. The environment source code was expanded from approximately 800 lines to 1700 lines. These modifications provided better learning signals and helped the agent achieve stable driving behaviour.

Experimental results confirm the effectiveness of the proposed methodology. After training for 1.8 million timesteps, the agent achieved a best reward of 1070.93 and a final moving average reward of 919.15. The model also reached 100% track completion in final training episodes and successfully completed 982 laps during training.

Testing results further validated the performance of the trained agent. The model completed all evaluation tracks with an average reward of 936.42 and 100% completion rate across three test episodes. This demonstrates that the learned policy generalizes well and can consistently perform autonomous racing tasks.

Overall, the project shows that reinforcement learning combined with environment engineering can effectively solve continuous control problems such as autonomous driving in simulation. Future improvements may include obstacle avoidance, multi-track generalization, transfer learning, and deployment in realistic driving simulators**.**

9. Future Work

In future work, the proposed autonomous racing system can be extended in several directions to improve performance and realism. One possible improvement is training the agent in more complex environments containing obstacles, traffic vehicles, and dynamic road conditions. This would help the model learn collision avoidance and safer driving strategies.

Another important extension is multi-track generalization, where the agent is trained to perform well on different road layouts, weather conditions, and surface types without retraining. Domain randomization and transfer learning techniques can also be applied to improve adaptability.

More advanced reinforcement learning algorithms such as Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3), or hybrid model-based

methods may further improve sample efficiency and control smoothness.

The current system uses simulation only. In future, the trained policy can be integrated with realistic simulators such as CARLA or AirSim, and later adapted for small robotic vehicles or real autonomous platforms.

Additional improvements may include camera-based perception, lane detection, sensor fusion, energy-efficient driving, and multi-agent racing scenarios. These extensions can make the proposed system more practical for real-world autonomous driving research.

10. References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.

[2] G. Brockman et al., "OpenAI Gym," arXiv preprint arXiv:1606.01540, 2016.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.

[5] PyTorch Team, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.

[6] Farama Foundation, "Gymnasium Documentation: CarRacing Environment," 2024. Available: https://gymnasium.farama.org/environments/box2d/car_racing/

[7] OpenAI, "Gym: A toolkit for developing and comparing reinforcement learning algorithms," 2016. Available: https://github.com/openai/gym

[8] Farama Foundation, "Gymnasium: Standard API for reinforcement learning environments," 2022. Available: https://github.com/Farama-Foundation/Gymnasium

[9] O. Klimov, "CarRacing Environment," originally developed for OpenAI Gym and later maintained in Gymnasium. Available: https://github.com/Farama-Foundation/Gymnasium

[10] K. Arulkumaran et al., "Deep Reinforcement Learning: A Brief Survey," *IEEE Signal Processing Magazine*, 2017.

[11] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[12] S. Haykin, *Neural Networks: A Comprehensive Foundation*, Pearson Education, 1999.